

Industrial Yield Optimization Demo Project

Project Overview

This project is a demo industrial AI solution for:

- Manufacturing Yield Optimization
- Scrap Reduction
- Process Parameter Monitoring
- Cross-Plant Benchmarking
- AI-Based Setpoint Recommendation
- Semantic KPI Governance

The demo is designed for a bootcamp presentation for Carrier Global.

The solution simulates:

- Historian sensor data
- LIMS quality lab results
- Environmental sensor streams
- ML-based yield prediction
- Recommendation engine
- Interactive operations dashboard

Recommended Scope (Improved Version)

Core Modules

1. Manufacturing Data Simulation

Simulate:

- Temperature
- Pressure
- Humidity
- Machine speed
- Vibration
- Raw material quality
- Shift operator
- Energy consumption
- Yield percentage
- Scrap rate
- Downtime

2. Semantic Layer KPIs

Governed KPIs:

- Yield %
- Scrap %
- OEE
- Throughput
- Batch Quality Score
- Energy per Unit
- Defect Rate
- Adoption Rate of Recommendations
- Plant-to-Plant Benchmarking

3. AI Prediction Engine

Model predicts:

- Expected Yield
- Scrap Risk
- Recommended Process Settings

Algorithms:

- Random Forest Regressor
- XGBoost (optional)
- Isolation Forest for anomaly detection

4. Recommendation Engine

Suggest:

- Reduce pressure by X
- Increase line speed by Y
- Reduce humidity range
- Change operating temperature

Tracks:

- Operator adoption
 - Predicted improvement
 - Actual improvement delta
-

5. Interactive Dashboard

Pages:

- Executive Overview
 - Plant Monitoring
 - Yield Optimization
 - Recommendation Center
 - AI Insights
 - Benchmarking
 - Data Explorer
-

Recommended Tech Stack

Frontend Options

Option 1 (Recommended for Bootcamp)

Streamlit

Advantages:

- Very fast development
- Beautiful charts
- Easy ML integration
- Deployable free
- Great for demos

Libraries:

- streamlit
 - plotly
 - pandas
 - scikit-learn
-

Option 2

HTML + Flask

Advantages:

- More professional UI
- Better control over frontend

Disadvantages:

- More development effort

Recommended Final Choice

Use:

- Streamlit frontend
- Python backend
- CSV data simulation
- Scikit-learn ML
- Plotly charts

This is the fastest and best-looking approach for demo + portfolio.

Recommended Project Architecture

```
yield-optimization-ai/
|
├─ app.py
├─ requirements.txt
├─ README.md
├─ .gitignore
|
├─ data/
|   ├── raw/
|   │   └─ manufacturing_data.csv
|   |
|   ├── processed/
|   │   └─ processed_data.csv
|   |
|   └─ reference/
|       └─ semantic_kpis.csv
|
├─ models/
|   ├── train_model.py
|   ├── predict.py
|   ├── recommendation_engine.py
|   └─ saved_models/
|       └─ yield_model.pkl
|
```

```
├── simulator/
│   ├── generate_data.py
│   └── plant_simulator.py
├── dashboard/
│   ├── overview.py
│   ├── monitoring.py
│   ├── optimization.py
│   ├── benchmarking.py
│   ├── recommendations.py
│   └── insights.py
├── assets/
│   ├── logo.png
│   ├── style.css
│   └── background.jpg
├── utils/
│   ├── helpers.py
│   ├── charts.py
│   └── preprocessing.py
└── notebooks/
    └── experimentation.ipynb
```

Database Design

Main Table: manufacturing_data

Column	Description
timestamp	Production timestamp
plant_id	Plant identifier
production_line	Production line
batch_id	Batch number
temperature	Process temperature
pressure	Machine pressure
humidity	Environmental humidity
machine_speed	Line speed

Column	Description
vibration	Machine vibration
raw_material_quality	Raw material score
energy_consumption	Energy used
operator_shift	Shift name
yield_percentage	Final yield
scrap_rate	Scrap percentage
quality_score	Final quality score
downtime_minutes	Downtime
recommended_temperature	AI recommended temperature
recommended_pressure	AI recommended pressure
adoption_flag	Recommendation accepted
predicted_yield	ML prediction

Data Simulation Logic

Yield Formula Example

```
yield_percentage = (
    85
    + (raw_material_quality * 0.08)
    - (humidity * 0.05)
    + (temperature * 0.02)
    - (vibration * 0.2)
    + random_noise
)
```

Dashboard UI Layout

Sidebar

- Plant Selector
- Production Line Selector

- Date Filter
 - Shift Filter
-

Main Dashboard Sections

1. KPI Cards

Show:

- Average Yield
 - Scrap Rate
 - Production Efficiency
 - Energy Consumption
 - AI Adoption Rate
-

2. Live Process Monitoring

Charts:

- Temperature trend
 - Pressure trend
 - Yield trend
 - Scrap trend
-

3. AI Recommendation Panel

Example:

Recommendation:
Increase temperature by 3°C
Reduce humidity below 50%
Expected Yield Gain: +4.2%
Confidence Score: 91%

4. Plant Benchmarking

Compare:

- Plant A vs Plant B
- Yield

- Scrap
- Energy usage

Charts:

- Radar chart
 - Heatmap
 - Bar chart
-

5. AI Insights

Example insights:

- High vibration correlates with scrap increase
 - Humidity above 60% reduces yield by 8%
 - Night shift shows lower efficiency
-

Streamlit UI Suggestions

Make UI Professional

Use:

- Wide layout
 - Dark theme
 - Plotly charts
 - KPI metric cards
 - Tabs
 - Expanders
 - Progress bars
 - Animated gauges
-

Example App Flow

User Journey

1. Open dashboard
2. Select plant
3. View live KPIs
4. View predicted yield
5. Check AI recommendations

- 6. Compare plants
- 7. Export report

Recommended Visualizations

Visualization	Use
Line Chart	Sensor trends
Gauge Chart	Yield KPI
Heatmap	Correlation analysis
Scatter Plot	Process vs Yield
Bar Chart	Benchmarking
Sankey Diagram	Production flow
Radar Chart	Plant comparison

ML Workflow

Training Flow

```
Raw Data
↓
Preprocessing
↓
Feature Engineering
↓
Train/Test Split
↓
Random Forest Model
↓
Prediction
↓
Recommendation Engine
```

Recommendation Engine Logic

Example:

```
if humidity > 60:
    recommendation = "Reduce humidity below 55%"

if vibration > 8:
    recommendation = "Inspect machine bearings"
```

Sample requirements.txt

```
streamlit
pandas
numpy
plotly
scikit-learn
joblib
matplotlib
seaborn
faker
```

Phase-wise Development Plan

Phase 1

Setup

- Create folder structure
- Setup virtual environment
- Install dependencies
- Create GitHub repo

Phase 2

Data Simulation

- Create manufacturing simulator
 - Generate CSV data
 - Add multi-plant support
-

Phase 3

ML Model

- Train yield prediction model
 - Save model
 - Create inference pipeline
-

Phase 4

Dashboard

- Build KPI overview
 - Add filters
 - Add charts
 - Add recommendation panel
-

Phase 5

Advanced Features

- Plant benchmarking
 - AI insights
 - Semantic KPI layer
 - Report export
-

Suggested Demo Story

Business Problem

Manufacturing plants suffer from:

- High scrap rates
 - Yield variation
 - Unstable process settings
 - Operator dependency
 - Energy inefficiency
-

AI Solution

The platform:

- Continuously monitors plant data
 - Predicts yield performance
 - Recommends optimal process settings
 - Detects inefficiencies
 - Benchmarks plants globally
-

Business Impact

Potential impact:

- 5–12% yield improvement
 - 10–20% scrap reduction
 - Lower energy consumption
 - Faster operator decisions
 - Better process governance
-

Suggested Next Files To Build

Build in this order:

1. generate_data.py
2. manufacturing_data.csv
3. train_model.py
4. recommendation_engine.py

- 5. app.py
 - 6. dashboard modules
 - 7. styling improvements
-

Recommended Streamlit Multi-Page Structure

```
pages/  
|  
├─ 1_Overview.py  
├─ 2_Process_Monitoring.py  
├─ 3_Yield_Optimization.py  
├─ 4_Benchmarking.py  
├─ 5_AI_Insights.py  
└─ 6_Recommendations.py
```

Recommended UI Theme

Theme Style

Industrial AI + Smart Factory

Colors:

- Dark navy
- Cyan
- Electric blue
- Steel gray
- Neon highlights

UI Feel:

- Modern manufacturing command center
 - Industrial analytics platform
 - AI operations cockpit
-

Final Recommendation

For your bootcamp:

DO NOT overcomplicate.

The best approach:

- Strong UI
- Clean architecture
- Good storytelling
- Simulated industrial data
- Basic but functional ML
- Interactive dashboard

That is enough to make it look production-grade.

What We Should Build Next

Next step should be:

1. Generate realistic manufacturing dataset
2. Build ML prediction model
3. Create Streamlit dashboard
4. Add interactive recommendation engine
5. Add modern UI styling

This is the ideal order.